

Министерство образования и науки Кыргызской
Республики Кыргызский государственный технический университет
им. И. Раззакова
Факультет информационных технологий

Кафедра «Программное обеспечение компьютерных систем»

ОТЧЕТ

Тема: Система CRM для парикмахерской

Выполнила: Жусупекова Сезим

Проверила: Каримова Г.Т.

Бишкек-2025

ЛАБОРАТОРНАЯ РАБОТА №1

ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

Тема: Система CRM для парикмахерской

1. Описание системы

CRM-система для парикмахерской предназначена для автоматизации управления клиентами, записями, персоналом и услугами. Основные функции системы включают:

- Запись клиентов на услуги;
- Управление расписанием мастеров;
- Учет истории посещений клиентов;
- Управление услугами и ценами;
- Ведение клиентской базы и программы лояльности;
- Формирование отчетов по выручке и загрузке персонала;
- Автоматизированное напоминание клиентам о предстоящих визитах;
- Интеграция с платежными системами для онлайн-оплаты услуг.

Система состоит из четырех подсистем:

1. Подсистема управления записями клиентов;
2. Подсистема управления расписанием мастеров;
3. Подсистема финансового учета и расчетов;
4. Подсистема аналитики и отчетности.

2. Бизнес-процессы, подлежащие автоматизации

1. Запись клиента: Клиент выбирает услугу, дату, время и мастера. Запись фиксируется в системе.
2. Работа мастера: Мастер получает список назначенных клиентов и фиксирует выполнение услуг.
3. Оплата услуги: Клиент оплачивает услугу, информация о платеже заносится в систему.
4. Управление клиентами: Ведение базы клиентов, отправка уведомлений о предстоящих записях и акциях.
5. Анализ работы: Генерация отчетов о популярности услуг, загруженности персонала и выручке.
6. Программа лояльности: Начисление бонусов за услуги, предоставление скидок и акций постоянным клиентам.
7. Управление инвентарем: Контроль расходных материалов и автоматическое формирование заказов на закупку.

3. UML-диаграммы

3.1. Диаграмма классов

Основные классы системы:

- Клиент (ФИО, телефон, история посещений, баланс бонусов);
- Мастер (ФИО, специальность, расписание, рейтинг);
- Услуга (название, длительность, стоимость);
- Запись (дата, время, клиент, мастер, статус);
- Оплата (способ, сумма, дата);
- Администратор (управление записями, настройка услуг, просмотр отчетов);
- Расходные материалы (наименование, количество, поставщик, срок годности).

3.2. Диаграмма последовательности (пример записи клиента)

1. Клиент выбирает услугу и мастера.
2. Система проверяет доступные слоты в расписании.
3. Клиент подтверждает запись.
4. Система сохраняет запись и отправляет уведомление клиенту и мастеру.

3.3. Диаграмма вариантов использования

Акторы:

- Клиент (запись, просмотр истории, отмена записи);
- Мастер (просмотр расписания, изменение статуса записи);
- Администратор(управление записями, настройка услуг, просмотр отчетов, управление расходными материалами)

Ответы на вопросы:

1. Цели системы:

- Автоматизация управления клиентами, записями, персоналом и услугами.
- Упрощение записи клиентов и управления расписанием мастеров.
- Улучшение клиентского сервиса через напоминания и лояльность.
- Оптимизация финансового учета и отчетности.

2. Модульная структура системы:

- Подсистема управления записями клиентов.
- Подсистема управления расписанием мастеров.
- Подсистема финансового учета и расчетов.
- Подсистема аналитики и отчетности.

3. Основные информационные объекты системы CRM для парикмахерской:

- Клиент (ФИО, телефон, история посещений, баланс бонусов).
- Мастер (ФИО, специальность, расписание, рейтинг).
- Услуга (название, длительность, стоимость).
- Запись (дата, время, клиент, мастер, статус).
- Оплата (способ, сумма, дата).
- Администратор (управление записями, настройка услуг, просмотр отчетов).
- Расходные материалы (наименование, количество, поставщик, срок годности).

4. Порядок занесения студентов в БД (если заменить на клиентов парикмахерской):

- Регистрация клиента в системе (ввод ФИО, телефона).
- Создание записи в базе данных.
- При каждом посещении обновление истории посещений и начисление бонусов.

5. Функциональные характеристики системы:

- Запись клиентов на услуги.
- Управление расписанием мастеров.
- Ведение истории посещений клиентов.
- Управление услугами и ценами.
- Автоматизированные уведомления.
- Генерация отчетов.
- Интеграция с платежными системами.

6. Данные, поступающие на вход подсистемы контроля (успеваемости студентов → работы мастеров):

- Записи клиентов (ФИО, услуга, дата, время, мастер).
- Выполненные услуги.
- Отзывы клиентов.
- График работы мастеров.

7. Категории пользователей и их возможности:

- **Клиент:** Запись, просмотр истории, отмена записи.
- **Мастер:** Просмотр расписания, изменение статуса записи.
- **Администратор:** Управление записями, настройка услуг, просмотр отчетов, управление расходными материалами.

8. Назначение подсистемы обработки запросов, определения категории пользователей:

- Регистрация новых клиентов и мастеров.
- Определение уровня доступа пользователей.
- Обработка запросов на запись и изменения.
- Управление правами администраторов и сотрудников.

Лабораторная работа №2

1. Методика оценки диаграмм UML

$$S = \frac{\sum S_{об} + \sum S_{св}}{1 + Ч_{об} + \sqrt{T_{об}} + T_{св}}$$

- ◆ **Где:**
- *S* — итоговая оценка диаграммы.
 - *S_{об}* — сумма оценок для объектов на диаграмме (классы, акторы, варианты использования).
 - *S_{св}* — сумма оценок для связей на диаграмме (ассоциации, композиции, зависимости и т. д.).
 - *Ч_{об}* — общее число объектов на диаграмме.
 - *T_{об}* — число типов объектов (например, классы, атрибуты, методы).
 - *T_{св}* — число типов связей (ассоциация, агрегация, композиция, обобщение, зависимость).

2. Исходные данные для расчета

Рассмотрю две диаграммы UML для CRM-системы парикмахерской:

- 1 Диаграмма классов.
- 2 Диаграмма вариантов использования.

2.1. Диаграмма классов (Class Diagram)

Объекты:

Элемент	Количество	Оценка за 1 элемент	Сумма
Классы (Class)	7	5	35
Атрибуты (Attribute)	15	0.3	4.5
Операции (Operation)	10	0.3	3

◆ **Итого объектов:** *S*(об) = 35 + 4.5 + 3 = 42.5

Связи:

Связь	Количество	Оценка за 1 элемент	Сумма
Агрегация (Aggregation)	2	2	4
Композиция (Composition)	1	3	3
Обобщение (Generalization)	2	2	4
Зависимость (Dependency)	1	2	2

◆ **Итого связей:** $S(св) = 12 + 4 + 3 + 4 + 2 = 25$

Другие параметры:

- $Ч(об)$ (число объектов на диаграмме) = 7.
- $T(об)$ (число типов объектов) = 3 (классы, атрибуты, методы).
- $T(св)$ (число типов связей) = 5.

Расчет по формуле:

$$S(class) = (42.5 + 25)/(1 + 7 + \sqrt{3}) + 5)$$

$$S(class) = (67.5)/(1 + 7 + 1.732 + 5)$$

$$S(class) = (67.5)/(14.732) = 4.99$$

◆ **Оценка диаграммы классов: 4.99**

2.2. Диаграмма вариантов использования (Use Case Diagram)

Объекты:

Элемент	Количество	Оценка за 1 элемент	Сумма
Варианты использования (Use Case)	5	2	10
Актеры (Actor)	3	4	12

◆ **Итого объектов:** $S(об) = 10 + 12 = 22$

Связи:

Тип связи	Количество	Оценка за 1 связь	Сумма
Зависимость (Dependency)	4	2	8

◆ **Итого связей:** $S(св) = 8$

Другие параметры:

- $Ч_{об}$ (число объектов на диаграмме) = 5 + 3 = 8.
- $T_{об}$ (число типов объектов) = 2 (варианты использования, актеры).
- $T_{св}$ (число типов связей) = 1.

Расчет по формуле:

$$S_{\text{use case}} = \frac{22 + 8}{1 + 8 + \sqrt{2} + 1}$$

$$S_{\text{use case}} = \frac{30}{1 + 8 + 1.414 + 1}$$

$$S_{\text{use case}} = \frac{30}{11.414} = 2.63$$

◆ **Оценка диаграммы вариантов использования: 2.63**

3. Итоговые оценки и выводы

Диаграмма	Рассчитанная оценка	Оптимальный диапазон
Диаграмма классов	4.99	5.5 – 5
Диаграмма вариантов использования	2.63	3.5 – 4

1. Каково назначение языка UML?

UML используется для **моделирования, проектирования и визуализации** систем, обеспечивая стандартизированные способы описания структуры и поведения системы.

2. Перечислите основные диаграммы UML.

- **Статические:** диаграмма классов, объектов, компонентов, развертывания, пакетов.
- **Динамические:** диаграмма последовательности, коммуникации, состояний, активности, временная диаграмма.

3. Какие типы связей знаете?

- **Ассоциация:** связь между объектами.
- **Агрегация:** “слабое” целое.
- **Композиция:** “сильное” целое.
- **Обобщение:** наследование.
- **Зависимость:** зависимость одного объекта от другого.
- **Реализация:** связь между интерфейсом и его реализацией.

4. Для чего используется методика количественной оценки диаграмм?

Для **оценки качества** диаграмм, выявления сложностей и ошибок, улучшения проектирования и поддержки системы.

Лабораторная работа №3



1 Основные классы

В диаграмме у нас есть **семь** основных классов:

1. Клиент (Client)

- Описывает посетителя салона.
- Содержит информацию: id, name, phone, email.
- **Связи:**
 - Один клиент может иметь **несколько записей** на услуги.
 - У клиента может быть **программа лояльности**.

2. Сотрудник (Employee)

- Представляет сотрудника (парикмахера, администратора и т. д.).
- Данные: id, name, role.
- **Связи:**
 - Один сотрудник может быть привязан к **нескольким записям**.

3. Услуга (Service)

- Описывает услугу (стрижка, окрашивание и т. д.).
- Данные: id, name, price, duration.
- **Связи:**
- Услуга связана с **записями клиентов**.

4. Запись (Appointment)

- Фиксирует факт записи клиента.
- Данные: id, date, client_id, employee_id, service_id.
- **Связи:**
- Запись относится к **конкретному клиенту**.
- Запись привязана к **конкретному сотруднику**.
- Запись связана с **конкретной услугой**.
- Каждая запись **может иметь оплату**.

5. Оплата (Payment)

- Фиксирует факт оплаты за услугу.
- Данные: id, amount, date, appointment_id.
- **Связи:**
- Привязана к **конкретной записи**.

6. Программа лояльности (LoyaltyProgram)

- У клиента может быть бонусная программа.
- Данные: client_id, points, discounts.
- **Связи:**
- Один клиент может участвовать в **программе лояльности**.

7. Отчёты (Report)

- Используется для анализа работы салона.
- Метод +generateReport().
- **Связи:**
- Отчёты агрегируют данные **по записям и оплатам**.

2 Связи между классами

как эти классы **связаны** между собой.

- Client "1" -- "0..*" Appointment

Один клиент может иметь несколько записей.

- Employee "1" -- "0..*" Appointment

Один сотрудник обслуживает несколько записей.

- Service "1" -- "0..*" Appointment

Одна услуга может быть в нескольких записях.

- Appointment "1" -- "1" Payment

Каждая запись имеет ровно одну оплату.

- Client "1" -- "1" LoyaltyProgram

Каждый клиент может участвовать в программе лояльности.

- Report ..> Appointment и Report ..> Payment

Отчёты анализируют данные по записям и платежам.

3 Как это работает в реальной жизни?

1. Клиент записывается на услугу (создаётся объект Appointment).
2. Запись привязывается к сотруднику, услуге и времени.
3. Оплачивает услугу, создаётся объект Payment, который связан с записью.
4. Система может начислить бонусы (обновляются points в LoyaltyProgram).
5. Администратор может сформировать отчёт, анализируя записи и платежи (Report).

Ответы на контрольные вопросы

1. Каково назначение диаграмм классов?

Диаграммы классов предназначены для представления **структурной модели системы**, отображая классы, их атрибуты, методы и связи между ними. Они помогают разработчикам и аналитикам понять архитектуру системы, её компоненты и их взаимодействие.

2. Для чего используется диаграмма классов на стадии анализа?

На стадии анализа диаграмма классов помогает **выделить основные сущности предметной области**, их характеристики и связи между ними. Она упрощает процесс выявления требований и формирует основу для проектирования системы.

3. Для чего используется диаграмма классов на стадии проектирования?

На стадии проектирования диаграмма классов помогает **детализировать реализацию системы**, определить атрибуты и методы классов, установить точные связи между компонентами. Она также используется для генерации кода в объектно-ориентированном программировании.

4. Назовите основные компоненты диаграммы классов.

- **Классы** (объекты системы с атрибутами и методами)
- **Атрибуты** (данные, хранящиеся в классе)
- **Методы (операции)** (действия, выполняемые классом)
- **Связи между классами** (ассоциация, агрегация, композиция, обобщение)
- **Множественность** (указывающая количество экземпляров в связи)
- **Признаки видимости** (public, private, protected)

5. Назовите основные типы статистических связей между классами.

- **Ассоциация** – связь между двумя классами, показывающая, что один класс использует другой.
- **Агрегация** – слабая форма “целое-часть”, где части могут существовать отдельно.
- **Композиция** – сильная форма “целое-часть”, где части уничтожаются при удалении целого.
- **Обобщение (наследование)** – один класс наследует свойства и методы другого.

6. Что представляет собой ассоциация?

Ассоциация – это **связь между двумя классами**, которая показывает, что один класс каким-то образом **использует или взаимодействует** с другим. Например, класс “Клиент” может быть ассоциирован с классом “Запись”, так как клиент записывается на услугу.

7. В чём смысл множественности ассоциаций?

Множественность (multiplicity) указывает, **сколько экземпляров одного класса может быть связано с экземпляром другого класса**.

- **1..1** – один к одному (один сотрудник – один бейдж).
- **1.. или 1..N*** – один ко многим (один клиент – несколько записей).
- ***0.. или ***** – много ко многим (один сотрудник может обслуживать много клиентов, и клиент может обращаться к разным сотрудникам).

8. В чём отличие атрибутов от ассоциаций?

- **Атрибуты** – это **внутренние свойства** класса (например, имя клиента, его телефон).
- **Ассоциации** – это **связи между классами**, показывающие их взаимодействие (например, связь “Клиент – Запись”).

9. Что такое признак видимости?

Признак видимости определяет, **какие объекты могут обращаться к атрибутам и методам класса**. Основные типы:

- **+** (**public**) – доступен всем.
- **-** (**private**) – доступен только внутри класса.
- **#** (**protected**) – доступен внутри класса и его наследников.

10. Что представляет собой операции класса?

Операции (методы) – это **действия, которые может выполнять класс**. Например, в классе “Клиент” могут быть методы:

- ЗаписатьсяНаУслугу()
- Оплатить()

11. В чём смысл обобщения?

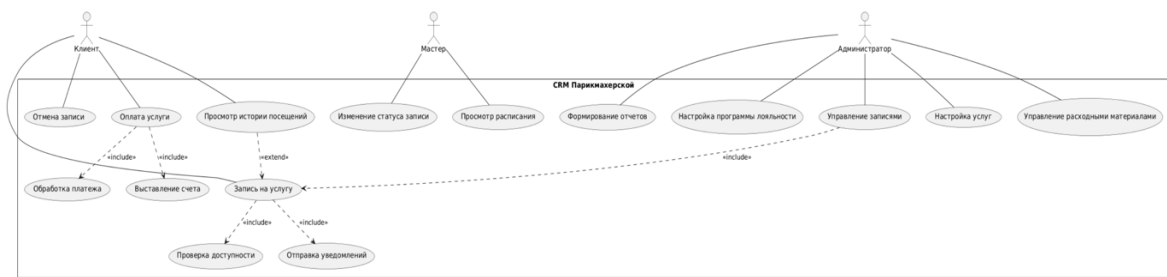
Обобщение (наследование) позволяет одному классу **унаследовать** атрибуты и методы другого класса. Это помогает **избежать дублирования кода**. Например:

- Класс “Парикмахер” наследует “Сотрудник”, потому что он – частный случай сотрудника.

12. Каково назначение ограничений на диаграммах классов?

Ограничения (constraints) задают **правила, которым должны соответствовать элементы системы**. Например:

- Возраст ≥ 18 – клиент может записаться только если ему 18+.
- Цена > 0 – услуга не может иметь отрицательную стоимость.



Расширенные сценарии вариантов использования

Запись на услугу

Актор: Клиент

Основной поток:

1. Клиент выбирает услугу и мастера.
2. Система проверяет доступность мастера и времени.
3. Клиент подтверждает запись.
4. Система фиксирует запись и отправляет уведомление клиенту и мастеру.

Альтернативные потоки:

- **3а.** Если время недоступно, система предлагает другие варианты.
- **4а.** Если клиент отменяет запись, система уведомляет мастера.

Оплата услуги

Актор: Клиент

Основной поток:

1. Клиент выбирает метод оплаты (карта, наличные, бонусы).
2. Система выставляет счет.
3. Клиент подтверждает оплату.
4. Система обрабатывает платеж и фиксирует его.

Альтернативные потоки:

- **3а.** Если оплата не проходит, система предлагает повторить попытку.
- **4а.** Если клиент использует бонусы, система пересчитывает сумму.

Управление записями (Администратор)

Актор: Администратор

Основной поток:

1. Администратор открывает список записей.
2. Может создать, изменить или удалить запись.
3. При изменении система уведомляет клиента и мастера.

Альтернативные потоки:

Если клиент не приходит, администратор может пометить запись как “неявка”.

Для количественной оценки диаграммы вариантов использования можно использовать разные метрики, такие как:

1 Количество актёров

2 Количество вариантов использования (Use Cases)

3 Количество связей (include, extend, ассоциации)

4 Глубина вложенности и сложность диаграммы

Количественная оценка диаграммы

Метрика	Значение
Количество актёров	3 (Клиент, Мастер, Администратор)
Количество вариантов использования	11
Количество связей include	5
Количество связей extend	1
Количество ассоциаций (обычных связей актёров с кейсами)	10
Общее количество связей	16
Среднее число связей на актёра	$\approx 5,3$
Среднее число связей на Use Case	$\approx 1,45$
Максимальная глубина вложенности (include)	2 (Оплата услуги → Выставление счета → Обработка платежа)

Анализ метрик

• Сбалансированность системы

→ Количество актёров и вариантов использования **сбалансировано**, все участники имеют по несколько действий.

• Нагрузка на актёров

→ **Клиент** выполняет больше всего действий (5), **Администратор** также имеет много задач (4), а **Мастер** только 2. Это логично, так как клиенты активно взаимодействуют с CRM.

- **Использование include и extend**

→ include — он описывает **обязательные шаги** (например, при оплате).

→ extend использован **один раз**, что указывает на **невысокую сложность и гибкость системы**.

- **Глубина вложенности**

→ **Максимальная вложенность – 2 уровня** (например, платежи). Это говорит о достаточной детализации, но без чрезмерной сложности.

Итоговая количественная оценка

На основе метрик, можно сказать, что **диаграмма имеет среднюю сложность**:

Хорошо распределены роли.

Используется разумное число связей.

Глубина вложенности не перегружает диаграмму.

Оценка: 8/10

Качественная оценка диаграммы

1. Полнота и корректность

- Охвачены все основные функции системы: запись клиентов, управление расписанием, оплата, история посещений, администрирование.

- Все актёры системы присутствуют.

- Используются include и extend для логического разделения процессов.

Оценка: 9/10

2. Понятность и читаемость

- Структура логична: актёры сверху, варианты использования внизу.

- Линии связей не пересекаются слишком часто.

- Вложенность include не превышает 2 уровней.

Оценка: 8.5/10

3. Логичность и взаимосвязь элементов

- include показывает обязательные процессы (например, оплата включает обработку платежа).

- extend используется там, где действие необязательно (например, повторная запись после просмотра истории).

- Баланс ролей соблюден: клиент использует больше функций, мастер и администратор имеют логичные обязанности.

Оценка: 9/10

4. Практическая применимость

- Диаграмма понятна разработчикам и бизнес-аналитикам.

- Можно добавить extend для возврата средств при отмене записи.

- Роль мастера можно расширить (например, добавлением комментариев к записям).

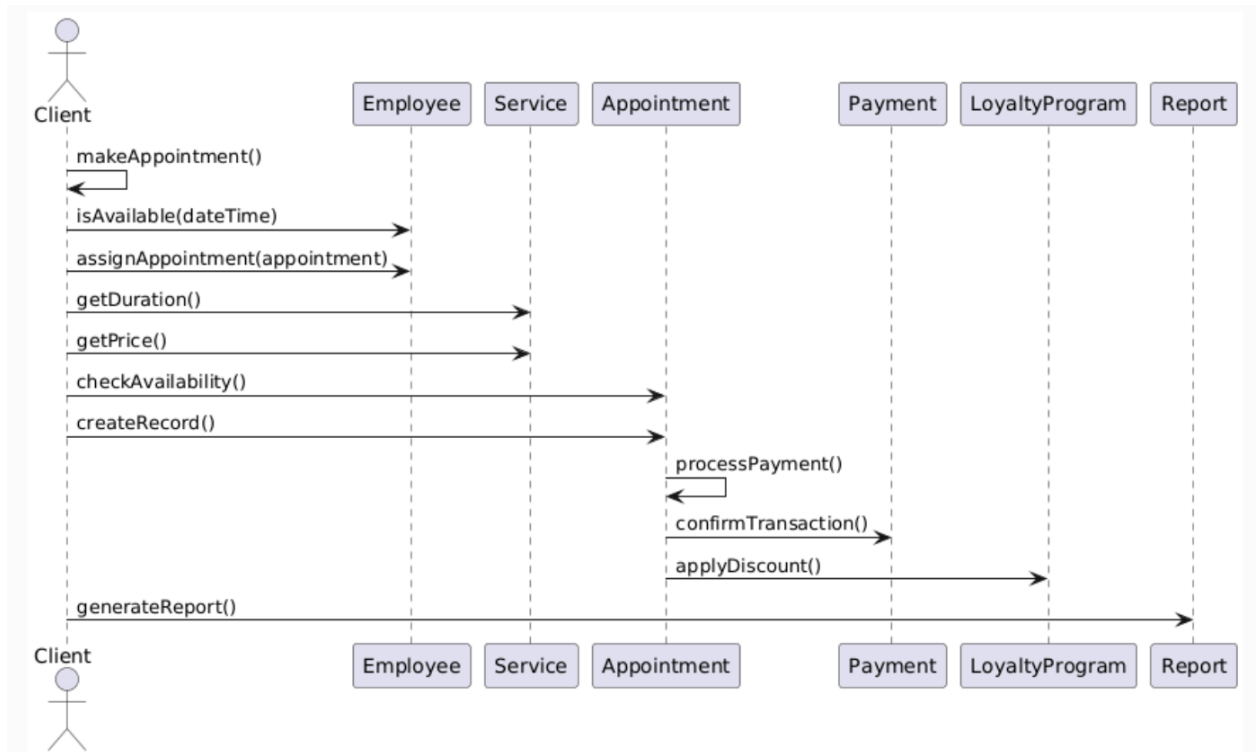
Оценка: 8.5/10

Общая оценка: 8.8/10

Диаграмма хорошо структурирована, логична и понятна. Возможны небольшие улучшения в детализации ролей и связей.

ЛАБОРАТОРНАЯ РАБОТА № 5

Диаграммы взаимодействия



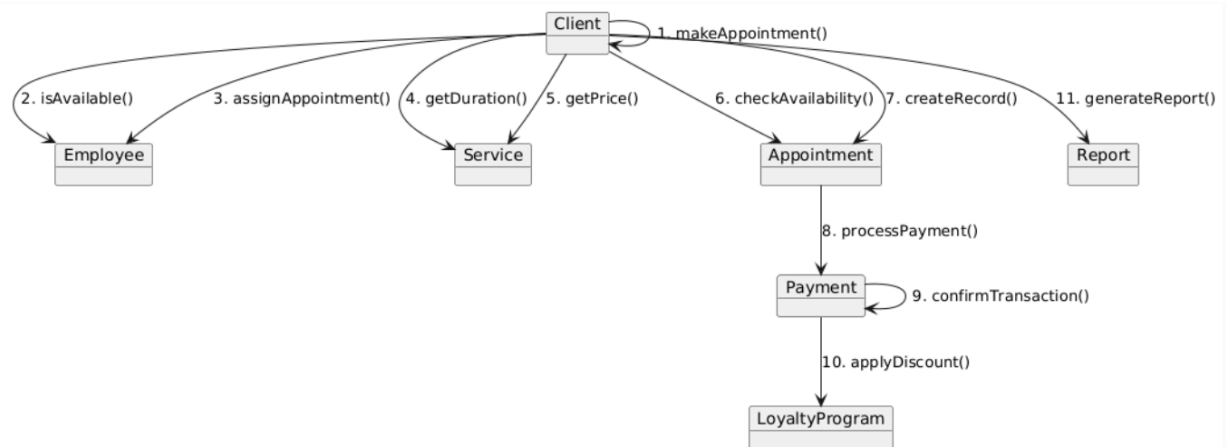
Назначение:

На диаграмме взаимодействия представлена последовательность действий между основными участниками системы в процессе записи клиента на услугу.

1. **Клиент** инициирует процесс записи через метод `makeAppointment()`.
2. Система обращается к объекту **Employee** (мастер), вызывая метод `isAvailable(dateTime)`, чтобы убедиться в его доступности на выбранное время.
3. Если мастер свободен, вызывается метод `assignAppointment(appointment)` — мастер прикрепляется к будущей записи.
4. Далее обращение идёт к **Service**, откуда запрашиваются параметры услуги: продолжительность (`getDuration()`) и стоимость (`getPrice()`).
5. После сбора информации создаётся объект записи через метод `Appointment.createRecord()`.
6. Затем проверяется её корректность (`checkAvailability()`) и запускается процесс оплаты (`processPayment()`).
7. Внутри оплаты вызывается метод `confirmTransaction()` у объекта **Payment** — он отвечает за завершение финансовой операции.
8. Параллельно с оплатой система инициирует начисление бонусов или скидков через метод `applyDiscount()` у **LoyaltyProgram**.

9. По завершении всех операций **Client** или администратор может запросить формирование отчёта через метод `generateReport()` у **Report**.

Кооперативная диаграмма



Кооперативная диаграмма отображает взаимодействие между объектами в виде сообщений, проходящих по структурным связям. Она делает акцент на **архитектуру и связи между объектами**, а не на временной порядок, как диаграмма последовательности. В отличие от диаграммы последовательности, кооперативная диаграмма делает акцент не на времени, а на **структуре и направлении связи между объектами**.

Краткая расшифровка взаимодействий:

1. **Client** → **Client** — метод `makeAppointment()` инициирует запись клиента.
2. **Client** → **Employee** — проверяется доступность мастера (`isAvailable()`), затем мастер назначается на услугу (`assignAppointment()`).
3. **Client** → **Service** — запрашиваются параметры услуги: длительность (`getDuration()`), стоимость (`getPrice()`).
4. **Client** → **Appointment** — создаётся запись (`createRecord()`), после предварительной проверки доступности (`checkAvailability()`).
5. **Appointment** → **Payment** — осуществляется процесс оплаты (`processPayment()`), подтверждение транзакции (`confirmTransaction()`).
6. **Payment** → **LoyaltyProgram** — начисление бонусов или скидок через `applyDiscount()`.
7. **Client** → **Report** — клиент или администратор может сформировать отчёт (`generateReport()`).

Количественная и качественная оценка диаграмм взаимодействия для CRM-системы парикмахерской

1 Диаграмма последовательности

Элемент	Кол- во	Ве с	Сумм а
Участники (объекты)	6	1	6
Сообщения	7	1. 5	10.5
Типы сообщений	1	1	1
Итого:			17.5

Расчёт:

$$S = (6 + 10.5 + 1) / (1 + \sqrt{6} + \sqrt{1}) = 17.5 / (1 + 2.45 + 1) \approx 17.5 / 4.45 \approx 3.93$$

Оценка диаграммы последовательности: ~3.93

2 Кооперативная диаграмма

Элемент	Кол- во	Ве с	Сумм а
Объекты	6	1	6
Сообщения	6	1. 5	9
Типы сообщений	1	1	1
Итого:			16

Расчёт:

$$S = (6 + 9 + 1) / (1 + \sqrt{6} + \sqrt{1}) = 16 / 4.45 \approx 3.6$$

Оценка кооперативной диаграммы: ~3.6

Качественная оценка диаграмм взаимодействия

Критерий	Диаграмма последовательности	Кооперативная диаграмма
Полнота	9	8

Критерий	Диаграмма последовательности	Кооперативная диаграмма
Понятность и читаемость	8.5	9
Логичность связей	9	9
Практическая применимость	8.5	8.5
Средняя оценка	8.75 / 10	8.6 / 10

Общий вывод:

Обе диаграммы соответствуют требованиям: они охватывают ключевые процессы CRM-системы (запись, оплата, бонусы), включают нужные объекты (Client, Appointment, Payment, LoyaltyProgram) и построены на основе классов, использованных во 2-й лабораторной. Диаграммы логичны, легко читаемы и пригодны для проектирования реальной системы.

Контрольные вопросы

1. Каково назначение диаграмм взаимодействия?

Диаграммы взаимодействия предназначены для моделирования динамического поведения системы. Они показывают, как объекты обмениваются сообщениями при выполнении конкретных задач. Это позволяет отследить последовательность операций и выявить взаимосвязи между компонентами системы.

2. Как относятся между собой диаграмма вариантов использования и диаграммы взаимодействия?

Диаграмма вариантов использования описывает, **что делает система с точки зрения пользователя**, а диаграммы взаимодействия уточняют, **как именно реализуется функциональность**, указанная в use-case. Иными словами, диаграммы взаимодействия детализируют реализацию вариантов использования.

3. Назовите два вида диаграмм взаимодействия.

- Диаграмма последовательности (Sequence Diagram)
- Кооперативная диаграмма (Communication Diagram)

4. Что такое «Жизненная линия» на диаграмме последовательности?

Жизненная линия (lifeline) представляет существование объекта во времени. Она изображается вертикальной пунктирной линией под объектом и показывает, когда объект существует и участвует в обмене сообщениями.

5. Как на диаграмме последовательности предоставляются сообщения?

Сообщения отображаются стрелками между жизненными линиями объектов.

- Синхронные вызовы: →
- Асинхронные вызовы: →→
- Ответы: ←←
- Также могут быть условные и циклические блоки (alt, opt, loop).

6. Что такое самоделегирование?

Самоделегирование — это ситуация, когда объект вызывает собственный метод, т.е. отправляет сообщение самому себе. Например: Client → Client : validateInput()

7. Что показывает активация объекта?

Активация объекта обозначается узким прямоугольником на жизненной линии и показывает **период выполнения операции** или метод, который активен у объекта в данный момент.

8. В чём отличие кооперативных диаграмм от диаграммы последовательности?

Кооперативные диаграммы делают акцент на **структуре и связях** между объектами и нумерации сообщений, а диаграммы последовательности — на **временном порядке** выполнения сообщений.

9. Каковы преимущества и недостатки каждого вида взаимодействия?

Тип диаграммы	Преимущества	Недостатки
Последовательность и	Чётко виден порядок сообщений, удобно анализировать процессы	Сложна при большом числе объектов
Кооперативная	Хорошо показывает структуру и взаимосвязи	Меньше наглядности по времени

10. Как отображается условное поведение на диаграммах взаимодействия?

Условное поведение отображается с помощью блоков alt, opt, loop:

- alt — альтернативные ветви (if-else)
- opt — опциональные действия
- loop — циклы (for, while)

ЛАБОРАТОРНАЯ РАБОТА №6

Диаграммы состояний

1. Диаграмма состояний для объекта "Запись" (Appointment)

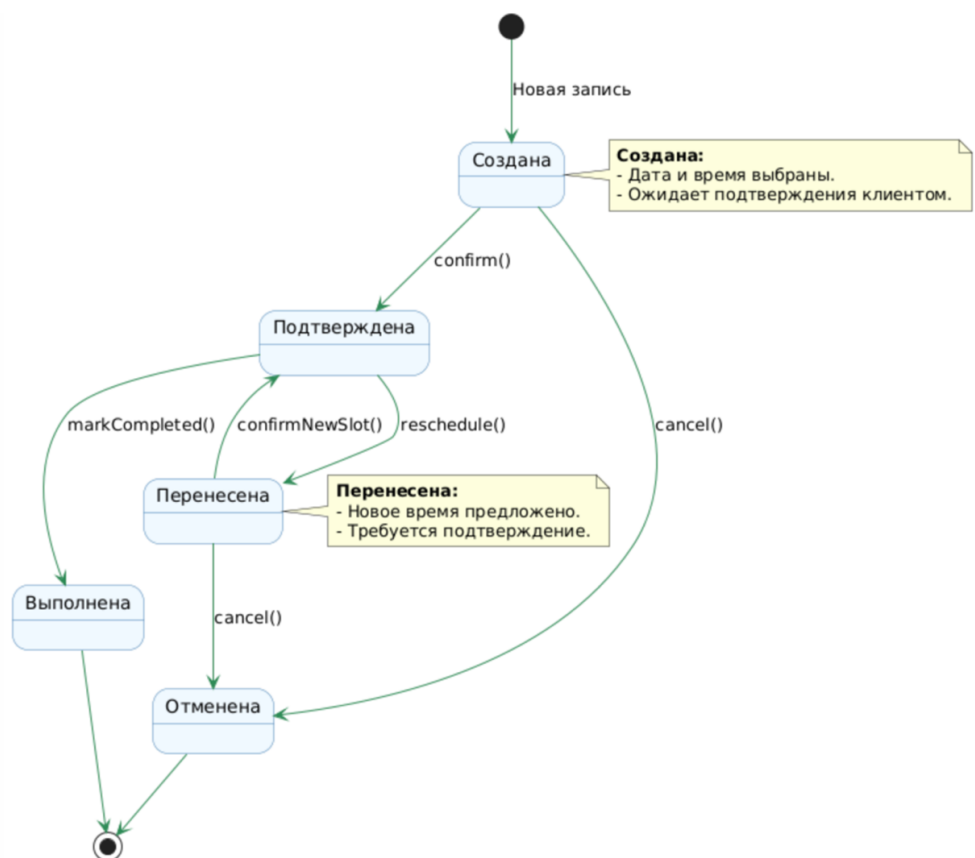
Цель: Моделирование жизненного цикла записи клиента на услугу.

Состояния объекта:

1. **Создана** — запись добавлена в систему, но не подтверждена.
2. **Подтверждена** — клиент подтвердил запись.
3. **Выполнена** — услуга оказана, запись закрыта.
4. **Отменена** — запись отменена клиентом или администратором.
5. **Перенесена** — запись изменена на другое время/дату.

События и переходы:

- **Создана → Подтверждена:** Клиент подтверждает запись.
- **Создана → Отменена:** Клиент/администратор отменяет запись.
- **Подтверждена → Выполнена:** Мастер отмечает услугу как выполненную.
- **Подтверждена → Перенесена:** Клиент запрашивает перенос.
- **Перенесена → Подтверждена:** Новое время подтверждено.
- **Перенесена → Отменена:** Клиент отменяет перенесенную запись.



3. Количественная оценка

Состояния: 5.

Переходы: 7.

Типы событий:

- confirm()
- cancel()
- markCompleted()
- reschedule()
- confirmNewSlot()

Расчет по формуле:

$$S = \frac{\sum S_{\text{сост}} + \sum S_{\text{переход}}}{1 + N_{\text{сост}} + \sqrt{N_{\text{типов}}}}$$

$$S = \frac{(5 \times 5) + (10 \times 2)}{1 + 5 + \sqrt{3 + 2}} = \frac{25 + 20}{6 + 2.24} \approx \frac{45}{8.24} \approx 5.46$$

Оценка: 5.46

4. Качественная оценка

1. Полнота:

- Охвачены основные состояния (создание, подтверждение, выполнение, отмена, перенос).
- **Недостаток:** Нет учета статуса оплаты (например, "Ожидает оплаты").

2. Понятность:

- Цветовая маркировка переходов улучшает читаемость.
- Пояснительные заметки раскрывают логику состояний.

3. Логичность:

- Переходы соответствуют бизнес-процессам парикмахерской.
- **Рекомендация:** Добавить состояние "Ожидает оплаты" и связь с оплатой.

4. Практическая применимость:

- Диаграмма может быть использована для разработки логики системы.
- **Недостаток:** Нет обработки повторных записей после отмены.

Ответы на контрольные вопросы:

1. Назначение диаграмм состояния

Диаграммы состояния в UML используются для моделирования **жизненного цикла объекта** в системе. Они показывают:

- Какие **состояния** может принимать объект.
- Как объект переходит между состояниями под воздействием **событий**.
- Какие **действия** выполняются при входе/выходе из состояния или во время пребывания в нём.

Пример: Для объекта "Запись в парикмахерской" диаграмма состояний отображает переходы:

Создана → Подтверждена → Выполнена/Отменена.

2. Отображение действий и деятельности

- **Действия (Actions):**

Краткие операции, выполняемые **при переходе между состояниями** или **в момент входа/выхода** из состояния.

- Обозначаются ключевыми словами:
 - entry / — действие при входе в состояние.
 - exit / — действие при выходе из состояния.
 - do / — деятельность, выполняемая **пока объект находится в состоянии**.

Пример:

```
state "Подтверждена" as Confirmed {  
    entry / sendNotification()  
    exit / logStatusChange()  
    do / checkPayment()  
}
```

- **Деятельности (Activities):**

Длительные процессы, выполняемые **внутри состояния**. Например, ожидание оплаты или обработка данных.

3. Условный переход

Это переход между состояниями, который происходит **только при выполнении условия**.

- **Синтаксис в UML:**[условие]
- **Пример:**
 - state "Ожидает оплаты" as PendingPayment
 - PendingPayment --> Paid : [оплата успешна] processPayment()
 - PendingPayment --> Canceled : [оплата не прошла] cancel()

4. Особые состояния объекта

- **Начальное состояние (Initial State):** Точка входа в жизненный цикл объекта. Обозначается **чёрной точкой (●)**.
- **Конечное состояние (Final State):** Точка выхода, где жизненный цикл объекта завершается. Обозначается **кружком с точкой внутри (◎)**.
- **Историческое состояние (History State):** Позволяет объекту вернуться к последнему активному состоянию после прерывания. Обозначается **Н**.

Пример:

```
[*] --> Created : Начало  
Completed --> [*] : Конец
```

5. Преимущества и недостатки диаграмм состояния

Преимущества	Недостатки
Наглядность жизненного цикла объекта.	Сложность при большом числе состояний.
Помогают выявить ошибки логики (например, незавершенные переходы).	Перегруженность диаграммы при детализации.
Упрощают проектирование сложных систем с множеством состояний (например, заказы, платежи).	Трудно моделировать параллельные процессы.

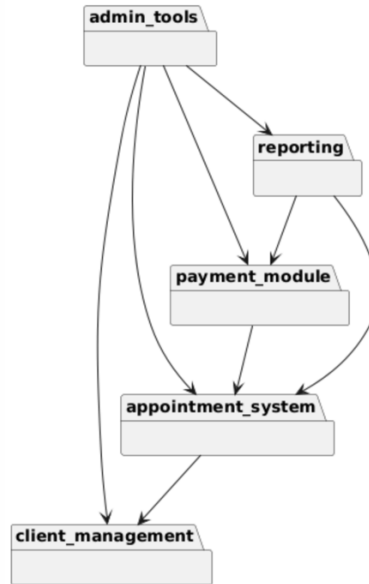
Пример недостатка:

Для объекта "Запись" с состояниями Создана, Подтверждена, Оплачена, Выполнена, Отменена диаграмма может стать слишком громоздкой, если добавить подсостояния (например, Оплачена → Ожидает подтверждения банка).

ЛАБОРАТОРНАЯ РАБОТА № 7

Тема: Диаграммы пакетов, компонентов и размещения для CRM-системы парикмахерской

Диаграмма пакетов



1. Диаграмма пакетов

Назначение:

Диаграмма пакетов показывает логическую группировку классов и модулей по функциональным подсистемам. Это позволяет структурировать проект и упростить сопровождение.

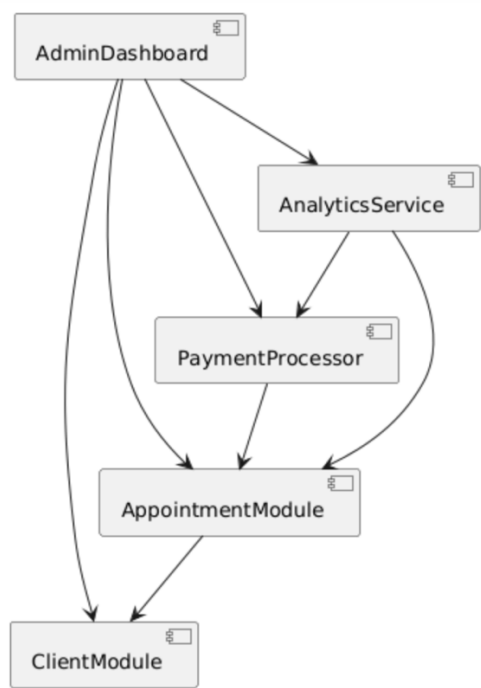
Описание пакетов:

Пакет	Назначение
client_management	Управление клиентами и программой лояльности
appointment_system	Обработка записей, расписания мастеров и услуг
payment_module	Механизмы обработки платежей и финансовых операций
reporting	Генерация аналитических отчетов
admin_tools	Интерфейс администратора, настройки, управление системой

Взаимосвязи:

- appointment_system зависит от client_management
- payment_module зависит от appointment_system
- reporting использует данные из appointment_system и payment_module
- admin_tools управляет всеми подсистемами

Диаграмма компонентов



2. Диаграмма компонентов

Назначение:

Диаграмма компонентов показывает, как логические пакеты реализуются в виде программных компонентов, и как эти компоненты взаимодействуют.

Компоненты и реализация:

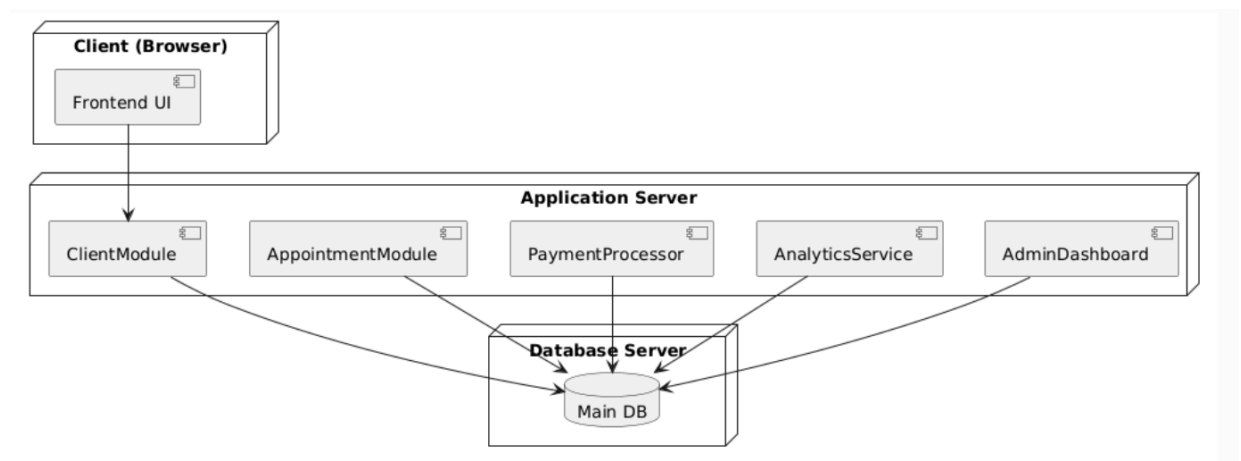
Компонент	Реализует пакет	Назначение
ClientModule	client_management	Работа с клиентами и программой лояльности
AppointmentModule	appointment_system	Управление расписанием и записями
PaymentProcessor	payment_module	Обработка оплаты
AnalyticsService	reporting	Создание аналитических отчётов

Компонент	Реализует пакет	Назначение
AdminDashboard	admin_tools	Интерфейс для администратора

Взаимосвязи компонентов:

- AppointmentModule использует ClientModule
- PaymentProcessor зависит от AppointmentModule
- AnalyticsService использует AppointmentModule и PaymentProcessor
- AdminDashboard взаимодействует со всеми остальными компонентами

Диаграмма размещения



Назначение:

Диаграмма размещения демонстрирует, как компоненты системы разворачиваются на физическом или виртуальном оборудовании.

Архитектура системы:

Узел (Node)	Размещённые компоненты
Client (Browser)	Frontend UI — интерфейс для клиентов и админов
Application Server	ClientModule, AppointmentModule, PaymentProcessor, AnalyticsService, AdminDashboard
Database Server	Main DB — централизованное хранилище данных

Взаимодействия:

- Frontend UI общается с ClientModule по API
- Все серверные модули подключаются к Main DB

Вывод

Построенные диаграммы позволяют:

- Увидеть **архитектуру системы на логическом и физическом уровнях**;
- Разделить проект на **подсистемы и компоненты**;
- Понять, **как развёрнута система** и как взаимодействуют её части

Количественная оценка UML-диаграмм

Используем формулу из методички:

$$S = \frac{S_{об} + S_{св}}{1 + Ч_{об} + \sqrt{T_{об}} + T_{св}}$$

Где:

- $S(об)$ — сумма баллов за объекты (пакеты, компоненты, ноды и т.п.)
- $S(св)$ — сумма баллов за связи (зависимости, размещение и т.д.)
- $Ч(об)$ — количество объектов
- $T(об)$ — количество типов объектов
- $T(св)$ — количество типов связей

Диаграмма пакетов

Объекты:

- Пакеты: $5 \times 3 = 15$

Связи:

- Зависимости: $7 \times 2 = 14$

Параметры:

- $S(об) = 15$, $S(св) = 14$
- $Ч(об) = 5$, $T(об) = 1$ (только пакеты)
- $T(св) = 1$ (один тип — зависимость)

Расчёт:

$$S = \frac{15 + 14}{1 + 5 + \sqrt{1} + 1} = \frac{29}{1 + 5 + 1 + 1} = \frac{29}{8} = 3.63$$

Диаграмма компонентов

Объекты:

- Компоненты: $5 \times 4 = 20$

Связи:

- Зависимости: $7 \times 2 = 14$

Параметры:

- $S(об) = 20$, $S(св) = 14$
- $Ч(об) = 5$, $T(об) = 1$
- $T(св) = 1$

$$S = \frac{20 + 14}{1 + 5 + \sqrt{1} + 1} = \frac{34}{8} = 4.25$$

Диаграмма размещения

Объекты:

- Ноды: $3 \times 4 = 12$
- Компоненты внутри: $6 \times 1.5 = 9$
- Всего: $S(\text{об}) = 12 + 9 = 21$

Связи:

- Связи между компонентами и БД: $6 \times 2 = 12$

Параметры:

- $Ч(\text{об}) = 9$, $Т(\text{об}) = 2$ (ноды, компоненты)
- $Т(\text{св}) = 1$

$$S = \frac{21 + 12}{1 + 9 + \sqrt{2} + 1} = \frac{33}{11.41} \approx 2.89$$

Качественная оценка

Критерий	Пакет ы	Компонент ы	Размещени е
Полнота	9	9	8
Понятность/читае- мость	8.5	9	8.5
Логичность связей	9	9	8.5
Практическая польза	8.5	9	8.5
Средняя оценка	8.75	9	8.375

Вывод

Диаграмма	Количественная оценка	Качественная оценка
Диаграмма пакетов	3.63	8.75
Диаграмма компонентов	4.25	9
Диаграмма размещения	2.89	8.38

Ответы на контрольные вопросы:

1. Какую проблему проектирования призвана решить диаграмма пакетов?

Диаграмма пакетов помогает **структурировать проект**, разделяя систему на логические модули. Это облегчает:

- понимание архитектуры,
- разделение обязанностей между командами,
- повторное использование кода,
- управление зависимостями.

2. В чём отличие диаграммы пакетов UML от диаграммы классов?

- **Диаграмма классов** описывает **внутреннюю структуру системы** — классы, их атрибуты, методы и связи.
- **Диаграмма пакетов** показывает **внешнюю архитектурную структуру**, группируя классы в модули и отображая зависимости между ними.

3. В чём смысл зависимости между элементами диаграммы пакетов?

Зависимость указывает, что **один пакет использует элементы другого**. Это значит, что при изменении одного пакета может потребоваться пересборка или изменение зависимого пакета.

4. Что такое интерфейс класса?

Интерфейс класса — это **набор публичных методов**, которые доступны другим объектам. Интерфейс описывает **то, что класс делает**, но не **как он это делает**. Он позволяет использовать класс без знания его реализации.

5. По каким признакам классы группируются в пакеты?

Классы объединяются в пакеты по принципу:

- **Функциональности** (например, клиентская логика, платежи),
- **Области ответственности** (например, логика взаимодействия с БД),
- **Уровню доступа или слоям архитектуры** (например, UI, логика, доступ к данным).

6. Какие виды элементов модели представлены на диаграмме компонентов?

На диаграмме компонентов представлены:

- **Компоненты** (реализуют пакеты/подсистемы),
- **Интерфейсы** (точки взаимодействия между компонентами),
- **Зависимости** (один компонент использует другой).

7. Как связаны между собой диаграмма пакетов и диаграмма компонентов?

Диаграмма пакетов показывает **логическую структуру**, а диаграмма компонентов — **физическую реализацию этих пакетов**. Компоненты обычно **реализуют** или **содержат** содержимое соответствующих пакетов.

8. Что показывает диаграмма размещения?

Диаграмма размещения отображает, **как программные компоненты размещаются на физическом или виртуальном оборудовании**: серверах, базах данных, клиентских устройствах и т.д.

9. Какие сущности отображаются на диаграммах размещения?

На диаграмме размещения отображаются:

- **Узлы (Nodes)** — устройства/сервера, на которых развёрнута система,
- **Компоненты** — части приложения, работающие на этих узлах,
- **Связи** — коммуникационные каналы между узлами и компонентами.

10. В каких случаях необходимо применение диаграммы размещения?

Диаграмма размещения используется, когда нужно:

- Разработать **инфраструктуру развертывания** приложения;
- Понять **нагрузку на сервера**;
- Спланировать **безопасность и сетевую архитектуру**;
- Отразить **технические требования и взаимодействие системных компонентов**.